

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэкенд-разработка

Отчёт

Домашнее задание №2
«Проектирование и технический дизайн API»

Выполнил:
Клименков Владислав

Группа:
К3341

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задачи

- Описать все функциональные и нефункциональные требования к бэкенд-приложению.
- Выбрать формат реализации API: REST API или Backend For Frontend.
- Спроектировать все эндпоинты API в формате OpenAPI-схемы с учётом реализованной диаграммы БД.
- Описать тело каждого запроса и тело каждого ответа, продумать возможные ошибки, возвращаемые из API.

Ход работы

№1

В первую очередь были составлены следующие требования к бэкенд-приложению:

1. Управление пользователями (регистрация, аутентификация, профиль)

1.1. Регистрация нового пользователя

- Система должна предоставлять публичный эндпоинт для регистрации с обязательными полями: `first_name`, `last_name`, `username`, `email`, `password` и опциональным полем `bio`.
- При регистрации поле `role` должно автоматически устанавливаться в значение `USER`.
- Система должна проверять уникальность `username` и `email`.
- Пароль должен сохраняться в базе данных в хэшированном виде.
- После успешной регистрации в базе данных создаётся запись в таблице `User` с `is_verified = false`.

1.2. Подтверждение email (верификация)

- Система должна отправлять письмо со ссылкой для подтверждения на указанный `email`.
- При переходе по ссылке система должна устанавливать `is_verified = true`.
- Неверифицированные пользователи не могут выполнять вход в систему (или им ограничивается доступ к основным функциям).

1.3. Вход в систему

- Система должна предоставлять эндпоинт для входа по `username/email` и `password`.
- При успешной аутентификации система должна возвращать JWT-токен для доступа к защищённым ресурсам.
- Время жизни токена должно быть ограничено, с возможностью обновления через refresh-токен.

1.4. Личный кабинет пользователя

- Авторизованный пользователь должен иметь возможность просматривать и редактировать свои данные: `first_name`, `last_name`, `username`, `email`, `bio`.
- При изменении `email` система должна требовать повторную верификацию.
- Пользователь может изменить пароль, указав текущий и новый.

1.5. Роли пользователей

- Система поддерживает две роли: `USER` и `ADMIN`.
- Администратор имеет доступ к специальным эндпоинтам для управления любыми рецептами, комментариями, пользователями (блокировка, удаление).

2. Управление рецептами

2.1. Создание рецепта

- Авторизованный пользователь может создать новый рецепт, указав:
 - `title`,
 - `description`,
 - `dish_type_id` (ссылка на существующий тип блюда),
 - `difficulty` (ENUM: `HARD`, `MEDIUM`, `EASY`),
 - `is_published` (черновик или публикация).
- При создании рецепта автоматически проставляются `user_id` (текущий пользователь), `created_at`, `updated_at`.

2.2. Добавление ингредиентов к рецепту

- При создании или редактировании рецепта пользователь может указать список ингредиентов. Для каждого ингредиента указывается:
 - `ingredient_id` (выбор из справочника)
 - `quantity` (числовое значение)
 - `unit_id` (единица измерения из справочника)
 - `alt_unit_text` (альтернативный текст единицы, если требуется)
 - `preparation_note` (заметка по подготовке ингредиента)
- Система должна обеспечивать возможность создания нового ингредиента или единицы измерения, если их нет в справочнике (через отдельные административные эндпоинты или при добавлении).

2.3. Добавление шагов приготовления

- Для каждого рецепта можно добавить один или несколько шагов (RecipeStep), каждый из которых содержит:
 - `number` (порядковый номер)
 - `title`
 - `description`
- Шаги могут быть созданы сразу при создании рецепта или добавлены позже.

2.4. Добавление медиа к рецепту и шагам

- К рецепту можно добавить медиафайлы (фото/видео) через таблицу `RecipeMedia`. Каждое медиа имеет:
 - `sort_order` (порядок отображения)
 - `media_type` (`PHOTO` или `VIDEO`)
 - `media_url` (ссылка на файл)
- К каждому шагу также можно добавить медиа через `RecipeStepMedia` с аналогичными полями.
- Медиа могут быть добавлены при создании или позже, в том числе несколько на один шаг или рецепт.

2.5. Редактирование и удаление рецепта

- Авторизованный пользователь может редактировать только свои рецепты. Администратор может редактировать любой рецепт.
- При редактировании можно изменять любые поля рецепта, а также добавлять/удалять ингредиенты, шаги, медиа.
- Пользователь может удалить свой рецепт. При удалении рецепта должны каскадно удаляться связанные записи: ингредиенты рецепта, шаги, медиа, комментарии, рейтинги, сохранения.

2.6. Публикация и черновики

- Рецепт может находиться в статусе черновика (`is_published = false`) или быть опубликованным (`is_published = true`).
- Опубликованные рецепты видны всем пользователям (включая неавторизованных). Черновики видны только автору и администратору.

3. Социальные функции

3.1. Оценка рецептов (рейтинг)

- Авторизованный пользователь может поставить оценку рецепту от 1 до 5 (целое число) через таблицу `RecipeRating`.
- Пользователь может изменить свою оценку – старая запись обновляется.

- При удалении оценки запись из `RecipeRating` удаляется.
- Система должна агрегировать среднюю оценку рецепта для отображения на странице рецепта и в списках.

3.2. Комментарии к рецептам

- Авторизованный пользователь может оставить комментарий (`Comment`) к любому опубликованному рецепту.
- Комментарий содержит текст (`text`), даты `created_at`, `updated_at`.
- Пользователь может редактировать и удалять только свои комментарии.
- Администратор может удалять любые комментарии.

3.3. Лайки на комментарии

- Авторизованный пользователь может поставить лайк (`CommentLike`) любому комментарию.
- Один пользователь может поставить только один лайк на комментарий (уникальность пары `user_id + comment_id`).
- Повторный запрос на лайк от того же пользователя удаляет лайк (toggle).

3.4. Подписки на кулинаров (пользователей)

- Авторизованный пользователь может подписаться (`Subscription`) на другого пользователя.
- Подписка односторонняя: `subscriber_id` – кто подписывается, `subscribed_to_id` – на кого.
- Пользователь может отписаться, удалив соответствующую запись.
- В ленте рецептов (например, главная страница) должны показываться рецепты пользователей, на которых подписан текущий пользователь, в порядке убывания даты создания.

4. Поиск и фильтрация рецептов

4.1. Поиск по тексту

- Система должна предоставлять эндпоинт для поиска рецептов по подстроке в `title` и `description` (регистронезависимо).

4.2. Фильтрация по параметрам

- Фильтрация по типу блюда (`dish_type_id`) – выбор одного или нескольких типов.
- Фильтрация по сложности (`difficulty`) – выбор одного или нескольких уровней.
- Фильтрация по ингредиентам: возможность выбрать один или несколько ингредиентов (по `ingredient_id`). Система должна возвращать рецепты, в которых присутствуют все выбранные ингредиенты.

4.3. Комбинированный поиск

- Поиск и фильтры должны работать совместно (например, поиск по названию + фильтр по сложности и типу блюда).
- Результаты поиска должны быть отсортированы по дате создания (новые сверху) или по рейтингу (опционально).
- Должна быть реализована пагинация.

4.4. Поиск по автору

- Возможность найти все рецепты конкретного пользователя (по `user_id` или `username`).

5. Сохранение рецептов (закладки)

- Авторизованный пользователь может сохранить любой опубликованный рецепт в личный список “Сохранённые рецепты” (`SavedRecipe`).
- Сохранённый рецепт не дублируется, а создаётся связь между пользователем и рецептом с отметкой времени `saved_at`.

- Пользователь может просматривать список своих сохранённых рецептов.
- Повторное сохранение того же рецепта не создаёт дубликат, а удаляет сохранение (toggle).

6. Страница рецепта

- Отдельный эндпоинт для получения полной информации о рецепте по его ID:
 - Основная информация: заголовок, описание, тип блюда, сложность, автор, даты, рейтинг (средний рейтинг и количество голосов).
 - Список ингредиентов с количеством, единицами измерения и примечаниями.
 - Список шагов с номерами, заголовками, описаниями и прикрепленными медиа.
 - Список медиа, прикрепленных к рецепту в целом.
 - Список комментариев с информацией об авторе, дате, количестве лайков и отметке, лайкнул ли текущий пользователь.
- Если текущий пользователь авторизован, дополнительно возвращается: поставил ли он оценку, сохранил ли рецепт, подписан ли на автора.

7. Административные функции

- Администратор (роль ADMIN) имеет доступ к панели управления, где может:
 - Просматривать и удалять любые рецепты, комментарии.
 - Редактировать или удалять пользователей.
 - Управлять справочниками: DishType, Ingredient, Unit (CRUD).
 - Назначать роль ADMIN другим пользователям.

8. Вспомогательные требования

8.1. Валидация входных данных

- Все входные данные должны проходить валидацию на соответствие типам, длинам, обязательности полей.
- При нарушении валидации система должна возвращать понятные сообщения об ошибках с соответствующими HTTP-статусами (400 Bad Request).

8.2. Обработка ошибок и статусы

- При попытке доступа к несуществующему ресурсу (рецепт, пользователь и т.д.) должен возвращаться 404 Not Found.
- При попытке выполнения действия, на которое у пользователя нет прав (редактирование чужого рецепта, удаление чужого комментария), должен возвращаться 403 Forbidden.
- При попытке создания дублирующей записи (например, повторный лайк, повторная подписка) система может либо возвращать ошибку, либо игнорировать действие с соответствующим сообщением.

8.3. Аутентификация и безопасность

- Все эндпоинты, кроме регистрации, входа, получения списка рецептов и страницы рецепта, должны быть защищены JWT-токеном.
- Пароли ни при каких обстоятельствах не должны передаваться в открытом виде (кроме момента отправки на вход/регистрацию по HTTPS).
- JWT-токен должен содержать идентификатор пользователя и его роль.

№2

В качестве формата реализации API был выбран REST API, как более простой для базовой реализации и универсальный вариант по сравнению с Backend For Frontend.

№3, №4

Была составлена OpenAPI-схема, описывающая все эндпоинты API, тела запросов и ответов, а также возможные ошибки, возвращаемые из API. Данная OpenAPI-схема была размещена в соответствующем разделе репозитория.

Вывод

В рамках данной работы были осуществлены первичные проектирование и разработка технического дизайна API.